1. PROJECT TITLE

Digital Diary with Mood Tracking and Memory Features.

2. PROJECT DESCRIPTION

The "Digital Diary" is a personal journal application built using Python. It allows users to create daily diary entries, track their mood, search past entries, and view a calendar. It simulates a digital personal diary where users can record their day-to-day thoughts and emotions, ensuring a structured and retrievable memory system. The project focuses on core programming concepts like file handling, pattern matching using regular expressions, data organization using JSON, and time/date manipulation with `datetime` and `calendar` modules.

The diary is designed for use in online Python compilers, making it portable, lightweight, and easy to test or demonstrate in academic settings.

3. RESEARCH & BACKGROUND STUDY

In today's digital age, journaling has moved from paper to screens. Many users prefer typing and storing their thoughts electronically, allowing easy search and organization. Traditional applications often rely on cloud storage and advanced libraries, but the goal of this project was to simulate a basic diary using only standard Python libraries:

* “datetime” for managing dates

* “json” for structured data storage

* “r” for keyword search using regular expressions

* “calendar” for displaying current month calendar

The project was inspired by personal productivity tools and mental wellness tracking apps, aiming to provide a basic, secure, and accessible journal system without third-party dependencies.

4. SYSTEM DESIGN & IMPLEMENTATION

Modules and Libraries Used

* “datetime”: Captures the current date for entries and displays the calendar.

* “json”: Formats each entry in a structured and readable format.

* “re”: Enables searching through diary entries using keywords.

* “calendar”: Displays the current month and helps visualize dates.

Key Features Implemented

*Entry Creation: Users enter their mood, thoughts, and optional file attachments.

* Mood Tracking: Moods are stored with each entry and analyzed for frequency.

* Search Functionality: Uses regular expressions to find matching keywords in entries.

* Calendar View: Shows the current month using the built-in calendar module.

* In-Memory Data Storage: All entries are stored in a list (for online compiler compatibility).

Working Logic

* On each run, the user can choose from a menu to write a new entry, view previous entries by keyword, see a mood summary, or display the calendar.

* The project does not use file handling in this version due to online compiler restrictions.

5. TESTING & RESULTS

Test Cases Performed

Test Case	Description	Input Given	Expected Output	Result
TC1	Create a new diary entry	Mood: HappyEntry: Had a good day	Entry saved	✅ Pass
TC2	Search existing entry	Keyword: good	Displays the entry with the word	✅ Pass
TC3	Analyze mood frequency	Multiple entries with moods	Displays mood count summary	✅ Pass
TC4	View calendar	None	Current month calendar shown	✅ Pass
TC5	Invalid option from menu	Choice: 6	Shows error message	✅ Pass

6.CODE

```
import re

import calendar

from datetime import datetime


# Simulated diary storage (in place of file)

diary_entries = []


def create_entry():

    today = datetime.now().strftime("%Y-%m-%d")

    mood = input("How are you feeling today? ")

    content = input("Write your journal entry:\n")

    attachments = input("Add file paths (comma separated, optional): ").split(',')

    entry = {

        'date': today,

        'mood': mood,

        'content': content,

        'attachments': [a.strip() for a in attachments if a.strip()]

    }

    diary_entries.append(entry)

    print("✅ Entry saved!")


def search_entries():

    keyword = input("Enter a keyword to search in your entries: ").lower()

    found = False

    for entry in diary_entries:

        if (keyword in entry['content'].lower() or

            keyword in entry['mood'].lower() or
```

```
        keyword in entry['date']):
        print("\n📅 Date:", entry['date'])
        print("😊 Mood:", entry['mood'])
        print("📖 Entry:", entry['content'])
        print("📎 Attachments:", entry['attachments'])
        print("-" * 40)
        found = True
    if not found:
        print("❌ No entries found for that keyword.")

def show_calendar():
    now = datetime.now()
    year = now.year
    month = now.month

    # Get dates with entries for this month
    entry_days = set()
    for entry in diary_entries:
        entry_date = datetime.strptime(entry['date'], "%Y-%m-%d")
        if entry_date.year == year and entry_date.month == month:
            entry_days.add(entry_date.day)

    # Create a TextCalendar instance
    cal = calendar.TextCalendar(calendar.SUNDAY)
    cal_str = cal.formatmonth(year, month)

    # Modify calendar to mark entry days with '*'
```

```
    lines = cal_str.split('\n')
    for i in range(len(lines)):
    line = lines[i]
    new_line = ''
     for part in line.split():
     try:
    day = int(part)
     part = f"{day}*"
     if day in entry_days:
            new_line += part.rjust(4)
        except ValueError:
            new_line += part.rjust(4)
      if new_line.strip():
        lines[i] = new_line
    print("\n📅 Calendar for", calendar.month_name[month], year)
    print('\n'.join(lines))


def mood_analysis():
    moods = {}
    for entry in diary_entries:
      mood = entry['mood'].lower()
      moods[mood] = moods.get(mood, 0) + 1
    if moods:
      print("\n📊 Mood Analysis:")
      for mood, count in moods.items():
        print(f" - {mood.capitalize()}: {count} times")
    else:
      print("No entries yet to analyze.")
```

```
def main():
    while True:
        print("\n=== Digital Diary ===")
        print("1. Create New Entry")
        print("2. Search Past Entries")
        print("3. View Mood Analysis")
        print("4. Show Current Month Calendar")
        print("5. Exit")
        choice = input("Choose an option (1-5): ")
        if choice == '1':
            create_entry()
        elif choice == '2':
            search_entries()
        elif choice == '3':
            mood_analysis()
        elif choice == '4':
            show_calendar()
        elif choice == '5':
            print("Goodbye! 5️⃣")
            break
        else:
            print("❌ Invalid choice. Please enter a number between 1-5.")

if __name__ == "__main__":
    main()
```

7. OUTPUTS

```
=== Digital Diary ===
1. Create New Entry
2. Search Past Entries
3. View Mood Analysis
4. Show Current Month Calendar
5. Exit
Choose an option (1-5): 1
How are you feeling today? happy!
Write your journal entry:
the day was good.
Add file paths (comma separated, optional): day
✅ Entry saved!
```

```
=== Digital Diary ===
1. Create New Entry
2. Search Past Entries
3. View Mood Analysis
4. Show Current Month Calendar
5. Exit
Choose an option (1-5): 2
Enter a keyword to search in your entries: d

📅 Date: 2025-09-04
😊 Mood: happy!
📖 Entry: the day was good.
📎 Attachments: ['day']
```

```
=== Digital Diary ===
1. Create New Entry
2. Search Past Entries
3. View Mood Analysis
4. Show Current Month Calendar
5. Exit
Choose an option (1-5): 3

📊 Mood Analysis:
 - Happy!: 1 times
```

```
=== Digital Diary ===
1. Create New Entry
2. Search Past Entries
3. View Mood Analysis
4. Show Current Month Calendar
5. Exit
Choose an option (1-5): 4

📅 Calendar for September 2025
September2025
  Su  Mo  Tu  We  Th  Fr  Sa
   1   2   3  4*   5   6
   7   8   9  10  11  12  13
  14  15  16  17  18  19  20
  21  22  23  24  25  26  27
  28  29  30
```

8. CONCLUSION & FUTURE ENHANCEMENTS

Conclusion

The Digital Diary project successfully demonstrates how Python's standard libraries can be used to build a functional and interactive journal system. It meets the basic needs of journaling, mood tracking, and memory searching in a command-line interface, making it ideal for academic and personal learning purposes.

Future Enhancements

* Data Saving: Add file handling for local save/load.

* Encryption: Implement password-protected entries for privacy.

* GUI Version: Develop a Tkinter-based interface for ease of use.

* Graphical Mood Analysis: Use graphs to visualize mood trends over time.

* Export Entries: Allow exporting diary content to text or PDF.